

TTA + CSR Application Flow Chart

The whole system on five sheets: how a person gets in and what they can reach, how a request is actually gated, what the data looks like and where CSR hooks into it, the two operational chains and the join between them that *is* the product — and a complete inventory of every CSR file that has ever existed in either repository.

Sheet 01	Sheet 02	Sheet 03	Sheet 04	Sheet 05
Three doors, one system	Enforcement pipeline & the six seams	Data spine	The two chains	Complete file inventory

The whole application — three doors, one system



TTA STAFF SHELL /dashboard

Dashboard
/dashboard

all staff

Project Setup / Projects
/trials · /trials/:id

trials

REP Management
/rep-management

reps

Vendors
/vendors

vendors

Work Orders
/work-orders

workorders

Payments
/payments

payments

Banking + TDS
/bank-tds

payments

Reports (5 screens)
/reports/*

report_* keys

Courier
/courier

courier

User Management
/user-management

SUPER_ADMIN

Admin → Setup
/admin

config edit

CSR MODULE /csr (inside the same shell)

CSR Projects (list)
/csr

csr

Project Detail — 5 tabs
/csr/:id

csr

• Overview
work order shown as plain #id

• Contacts
client-side roster

• Activities
trial / workshop / training

• Reports
publish toggle

• Utilisation
expense tags + certificate

csr_certificate

CSR Clients (onboarding)
/csr/clients

ADMIN only

CSR Branding
/csr/branding

ADMIN only

Activity Types catalog
/csr/activity-types

ADMIN only

No Payments / Vendors / REP / Banking / Courier tab — absence is the scope boundary.

FUNDER PORTAL /client (own shell, read-only)

Branded login
/client/:slug/login · PUBLIC, pre-auth

Portal — 3 tabs
/client · role CSR_CLIENT

My Project
name · funder · sanctioned · dates · status

Activities
only those with a published report

Reports
only visible_to_client = true

Change password
self-serve — the G4 precondition

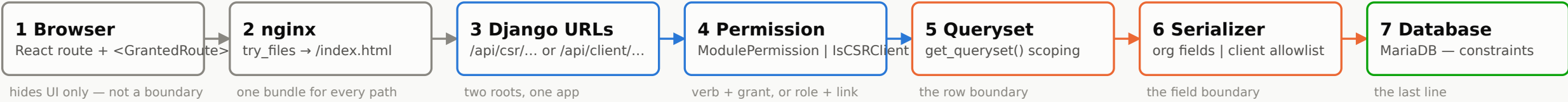
NEVER REACHES THE FUNDER
vendor names · payment rows · work order · contract
month-wise splits · expense tags · other projects
the Utilisation Certificate itself (no /api/client route)
— enforced by allowlist serializers + scoped querysets

BUT: SAME BUNDLE
build:client + ClientApp.jsx + client.html exist,
Dockerfile runs `npm run build` only, and nginx
sends every path to the same index.html.
The funder downloads the whole staff app. (G3)

Separation is by **route + sidebar + grant/role**, never by deployment. Internal staff are gated by **module grants** (no role); the external funder is gated by the **CSR_CLIENT role** plus a one-project link. SUPER_ADMIN bypasses every grant check.

How a request is actually gated — and where it leaks

A REQUEST, END TO END



WHAT EACH LAYER ACTUALLY ENFORCES

ModulePermission verb → grant. has_permission ONLY.	no row scoping · no field control · SUPER_ADMIN bypass
IsCSRClient role + resolvable project link, fail-closed	adds has_object_permission — the class ModulePermission never had
get_queryset() every client view filtered to ONE project	a guessed id returns 404, not 403 — existence is not confirmed
Allowlist serializer fields opted IN explicitly	a field absent from the list can never leak
DB constraints OneToOne(payment) unique · CHECK(xor)	CHECK is IGNORED on prod MariaDB 10.1.x — only the unique index holds

WHERE IT BREAKS — every defect sits BETWEEN two layers, not inside one

1 Serializer ↔ Database

paymentId declared explicitly → DRF drops UniqueValidator. Duplicate tag passes validation, dies at the index.

→ HTTP 500, no message. The audit rule the client stated as absolute is enforced by a crash.

2 Registry ↔ Button

csr_certificate is can_delete:false. The Utilisation tab still renders a delete icon.

→ 403 for every user except SUPER_ADMIN — who is the only one likely to test it.

3 Grant model ↔ Screen

The tag-expense picker reads /payment-requests/ (payments grant). csr_certificate unlocks no read on it.

→ 403 swallowed by .catch → empty dropdown, no error. The separation-of-duties operator cannot tag anything.

4 Build ↔ Deploy

ClientApp + build:client + client.html all exist; Dockerfile and nginx ignore them.

→ The funder's browser downloads the entire internal app. G3 called this information disclosure.

5 Report ↔ Portal

fileUrl is allow_blank. A blank-URL report can still be published.

→ The funder sees a report row and an activity row with nothing to open.

6 Docs ↔ Code

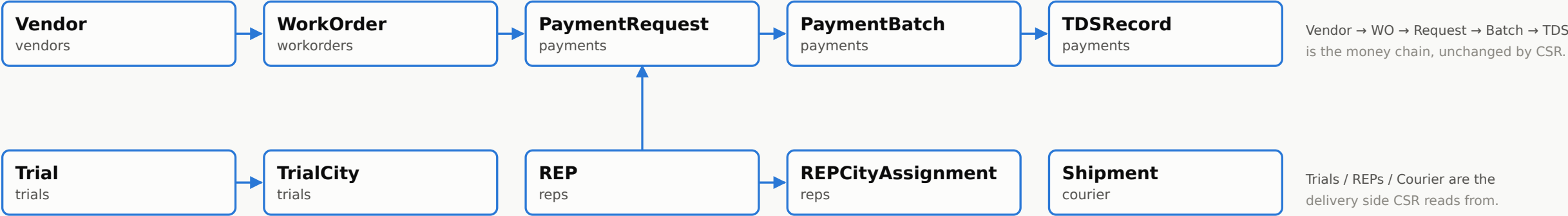
CSR_OPS was deleted from the planning docs but survives in the client-facing design review.

→ The client holds a document describing a role that never existed.

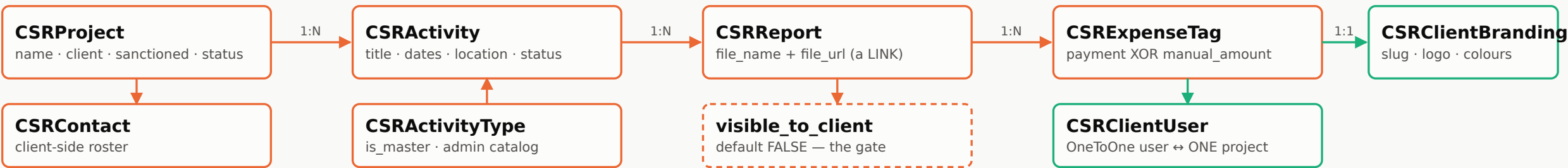
Each layer is competently built. **Nothing checks across the joins.** Every confirmed defect in this module lives in a seam between two correct layers.

The data spine — where CSR hooks into TTA

TTA CORE — the origin of all data



CSR APP — 7 new tables. Adds no data of its own to the core; only points at it.



THE ONLY THREE PLACES CSR TOUCHES TTA — everything else is read through these



MISSING FROM THE MODEL — and each absence blocks something that was asked for

- No vendor / partner FK on CSRActivity
 - a Workshop cannot record WHO delivered it → the partner tier has no edge to scope on
- No CSR_PARTNER role in User.ROLE_CHOICES
 - roles are SUPER_ADMIN · ADMIN · REP · CSR_CLIENT — the third audience does not exist
- No 'partner' vendor category in config
 - the design review scheduled it as build stage 3; it was never created
- No certificate snapshot / issuance record
 - the certificate is a live recompute — 'generated at project end' is not modelled

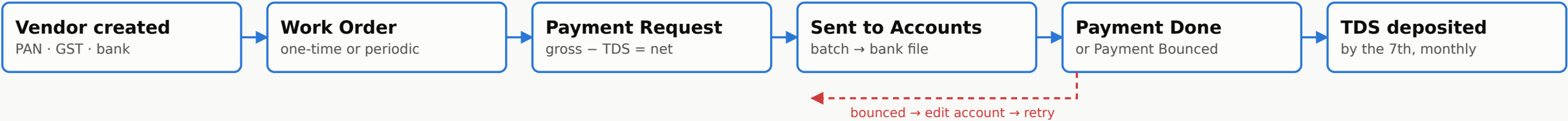
ONE-WAY RULE: csr may import trials / workorders / payments. They must NEVER import csr. Proven by a test that scans core apps for a reverse import (INV-DEP).

SINGLE DATABASE IS REQUIRED: cross-database joins are impossible, and the audit-uniqueness rule needs a DB-level unique index over one table.

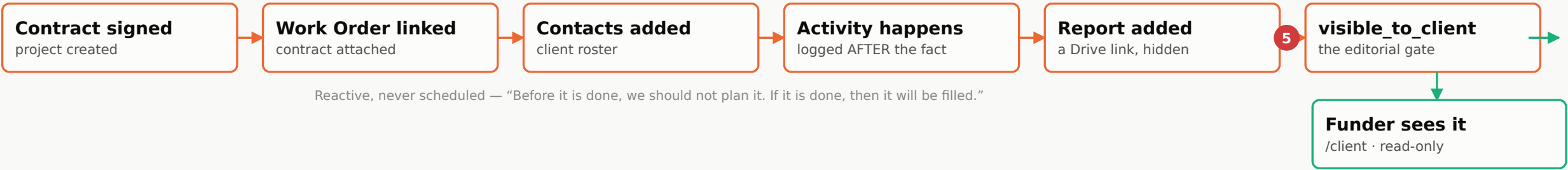
CSR holds **no copy** of a trial, vendor, work order or payment. An item appears in CSR only when someone deliberately tags it in. That is the client's one rule — “don't make a parallel system” — made structural.

The two chains, and the join that is the product

CHAIN A — THE MONEY (runs entirely in TTA; CSR never touches it)



CHAIN B — THE DELIVERY (what the funder is shown)



THE JOIN — where the two chains meet, and the only money decision inside CSR



Draft · Sent to Accounts · Bounced → shown in excludedItems, NOT counted
Manual amount → always counts (no status to check)

Gated by csr_certificate — a SEPARATE grant from csr.
“the payment will not be in the CSR... he will say, tomorrow you show it, how will you change it?”

THIS JOIN IS THE PRODUCT — AND IT IS THE PART THAT DOES NOT WORK

The operator granted csr + csr_certificate — the exact person the client asked to separate — gets 403 on /payment-requests/, swallowed by a .catch. The picker renders zero options under the helper text “A payment can be tagged to only one project.” They cannot tag anything, and are told nothing. They cannot untag either: DELETE is forbidden by the registry, and the button is rendered anyway. → no test, at any level, ever links a real payment.

THE CERTIFICATE AS A DOCUMENT

It is the deliverable — the artefact a funder files with a government audit. Generated: title, project, funder, sanctioned, total, a 3-column table. No letterhead · no signature block · no reference to the sanctioning contract · no period covered · no certificate number · no “we certify that...” clause. It is a live recompute, never frozen, with no record that one was ever issued. No planning document ever specified what it should contain.

THE ONE RULE, IN THE CLIENT'S OWN WORDS

“Let's say I have two CSR projects. They are showing it to you there and here too. So, it is a forgery, isn't it?
When the government audits, they cannot tell you how they have shown it twice. One person will always be unique.”

Every CSR file that has ever existed

Verified against the full git history of **both** repositories (`git log --all --name-only`) plus the working tree and `D:\CSR`. **Nothing has ever been deleted or renamed** — every CSR path that ever appeared in either repo still exists on disk today. Items marked **[uncommitted]** exist only in the working tree.

Backend — csr app (Django) · 21 files	
models.py	7 models: Project, ActivityType, Activity, Report, ExpenseTag, ClientUser, ClientBranding
views.py	8 org viewsets + onboarding + access revoke + Utilisation Certificate
serializers.py	org serializers (full fields)
urls.py	/api/csr/* (org) and /api/client/* (funder) — two roots, one app
client_views.py	funder surface — ReadOnly, queryset-scoped, fail-closed
client_serializers.py	G1 allowlist serializers — the field boundary
client_permissions.py	G2 IsCSRClient — role + project link, has_object_permission
certificate_rules.py	the shared counts_toward_certificate() predicate [uncommitted]
tests.py	51 tests · 16 classes
admin.py · apps.py · __init__.py	Django plumbing
migrations/0001 ... 0005	initial · report links · description+contacts · activity duration · branding
management/commands/seed_csr_demo.py	demo data seeder
Backend — outside the csr app	
accounts/models.py	User.ROLE_CHOICES gains CSR_CLIENT
accounts/migrations/0003_alter_user_role.py	the additive AlterField
accounts/token_utils.py	revoke_refresh_tokens() for off-boarding [uncommitted]
permissions/registry.py	csr + csr_certificate module grants
permissions/enforcement.py	EXTERNAL_ROLES — a funder is denied before any grant lookup
backend/settings.py · backend/urls.py	INSTALLED_APPS + api/ include
Frontend — components/csr · 17 files	
CSRProjectManagementPage.jsx	the project list
CSRProjectCard.jsx · CSRProjectDetailPage.jsx · CSRProjectDetailView.jsx	card → 5-tab detail → overview
CSRProjectModal.jsx · CSRActivityModal.jsx · CSRReportModal.jsx	create/edit modals
CSRContactModal.jsx · CSRExpenseTagModal.jsx	contacts · the tag-expense picker
CSRActivityTypesPage.jsx · CSRBrandingPage.jsx · CSRClientsPage.jsx	the three admin-only screens
index.js	barrel export
*.test.jsx ×5	Activity, Contact, Project, Report modal tests
Frontend — the funder portal (no 'csr' in the filename)	
components/client/ClientLogin.jsx	branded pre-auth login, by slug
components/client/ClientPortalPage.jsx	the whole portal — one page, three tabs
components/client/clientTheme.js	palette override only — brand is data
components/client/ClientChangePasswordDialog.jsx	the G4 auth precondition
ClientApp.jsx · client-index.js · public/client.html · craco.config.js	the G3 split build — built, never served
App.js · auth/roles.js · components/layout/Sidebar.jsx · services/api.js	routes, the CSR_CLIENT role, 2 nav items, csrAPI + clientAPI
Tests — e2e/csr · 8 files, committed	
tests/csr-operator.spec.js	S1-S10 operator journey · actor = ADMIN + csr + csr_certificate
tests/client-portal.spec.js	F1-F6 funder isolation, incl. 'no financial data anywhere'
playwright.config.js · package.json · scripts/bootstrap.sh · PLAN.md · README.md · .gitignore	harness

Planning & requirements · in the repo	
_docs/planning/CSR_COMPLETE_REFERENCE.md	GEN 4 — the single source of truth
_docs/planning/CSR_ARCHITECTURE.md	GEN 4 — guardrails G1-G4 + the invariant registry
_docs/planning/CSR_CLIENT_PORTAL.md	GEN 4 — white-label sub-spec
_docs/planning/CSR_PRODUCT_DESIGN.md	GEN 3 — first doc written against the code
_docs/planning/CSR_BRIEF.md	GEN 2 — merged the three GEN-1 readings
_docs/planning/CSR_MODULE_SPEC.md · CSR_IMPLEMENTATION.md · CSR_VISUAL_FLOW.md	GEN 1 — three parallel readings of the audio; none mentions partners
_docs/planning/CSR_SHELL_PLAN_2026-08-14.md	the shell plan [uncommitted]
_docs/requirements/README.md + transcripts/ ×4	the primary source [uncommitted]
_docs/review/2-2-csr-isolation.md, 4-1-silent-empty.md, FINDINGS.md ...	the 22-pass audit [uncommitted]
_docs/deployment/CLIENT_BUILD.md	G3 deploy instructions — the handoff

D:\CSR · the primary sources, outside git	
CSR_REC_1.m4a (176s)	“bifurcate the CSR view” — the partner ask
CSR_REC_2.m4a (76s)	two modules + RETF partners — LOST by the first transcription pass
WhatsApp 2026-05-22 7.26.43 PM (149s)	what a CSR project IS
WhatsApp 2026-05-22 7.27.20 PM (458s)	the money rule, the audit rule, the three routes
new_inputs/ 06-27 03.43.43 + 04.47.13 (543s)	85 SECONDS LONGER than the May original — never heard, not silence
CSR_Module_Design_Review.docx	the client-facing doc — still contains CSR_OPS; defines 'partner' as a VENDOR CATEGORY
transcrip[t/ ×3 generations + 4 zips	v1 (failed) · v1-retry · v2 large-v3
non-CSR/ ×4 recordings	owner-confirmed out of scope, deliberately never transcribed

Totals: backend 27 files · frontend 27 files · e2e 8 files · planning & requirements 20 files · D:\CSR 43 files across 9 directories. The entire requirement basis is **14 min 19 s** of phone audio, in one un-backed-up folder, 85 seconds of which has never been heard.